Information Retrieval

WS 2022 / 2023

Lecture 5, Tuesday November 22nd, 2022

(Fuzzy Search, Prefix Edit Distance, q-Gram Index)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Experiences with ES4 — Compression, Codes, Entropy
- Contents
  - Fuzzy search — type breifurg, find freiburg
  - Edit distance — similarity measure for strings
  - Fuzzy + q-grams — a fascinating relationship
  - Q-gram Index — tying it all together

ES5: Implement error-tolerant prefix search using a q-gram index and prefix edit distance

We have compiled a beautiful dataset for you: 2.6M entities from Wikidata with scores, descriptions and synonyms

Experiences with ES4 1/4

- Summary / excerpts
  - Relatively few problems with Exercise 1, relatively many problems with Exercise 2

"My experience with this sheet was quite satisfying"

"Mathematics is cool because its so hard in interesting ways"

"Math is a matter of willingness [...] more time is helpful"

"Very complicated task [...] spent 4 days trying to solve it"

"Needed a lot of time to show that $p \cdot i/\ln 2 \leq \log_2 1/(1-p)^{i-1}$

"I hope there will be more coding sheets and less math"

"Arithmetic expressions are simpler: progress is monotonic"

"Exercise more about math skills than about compression"

Experiences with ES4 2/4

You gave amazingly interesting feedback !

- Relationship with mathematics, what would help

  "I love mathematics, but am not good at it yet"

  "More examples with slow walk through what is happening"

  "Just need time to learn" "Time is the only thing that help"

  "I feel like I miss mathematical background"

  "Only few good teachers taught mathematical concepts intuitively"

  "My love for math increases <3"

  "It got and gets better with practicing"

  "Problems with logarithm rules … they feel so unintuitive"

  Very interesting bottom line: almost all of you actually love math, you just wish you had a better understanding of the basics, and more time to learn what you have maybe missed in the past

Experiences with ES4 3/4

$\forall x : 1 + x \le e^x$

$\log x^y = y \cdot \log x$

$\log_b a = \frac{\log a}{\log b}$

- Solution sketch for Exercise 2
  - The task was to prove $L_i \le \log(1/p_i) + 5$, where

    $L_i = \lfloor i / M \rfloor + 1 + \lceil \log_2 M \rceil$, $M = \lceil 1/p \cdot \ln 2 \rceil$, $p_i = (1 - p)^{i-1} \cdot p$
  - The obvious start is to write down the LHS (left-hand side) and RHS and plug in what we know and simplify

after the "simple" transformations: $\frac{(i-1) \cdot p}{\ln 2} \le \log_2 \frac{1}{(1-p)^{i-1}}$

$$\frac{(i-1) \cdot p}{\ln 2} \le \frac{(i-1) \cdot \ln \frac{1}{1-p}}{\ln 2}$$

$$p \le \ln \frac{1}{1-p}$$

$$e^p \le \frac{1}{1-p}$$

$$e^{-p} \ge 1 - p$$

BINGO!

not a proof yet

$1 - p \le e^{-p}$ [HINT]

$$\Rightarrow e^p \le \frac{1}{1-p}$$

$$\Rightarrow p \le \ln \frac{1}{1-p}$$

$$\Rightarrow (i-1) \cdot p \le \ln \frac{1}{(1-p)^{i-1}}$$

$$\Rightarrow \frac{(i-1) \cdot p}{\ln 2} \le \log_2 \frac{1}{(1-p)^{i-1}}$$

Experiences with ES4 4/4

- Math skills, my experience
  - My feeling is that it would help many of you to develop your **basic practical** math skills

    Unfortunately, one does not really learn these in the math lectures and that is probably a part of the problem
  - Actually, the **math skills** needed for computer science are not so different from **programming skills**:

    When you have a basic skill set, it's often just a matter of trying out a number of combinations until it works

    With more experience and clever ideas, you can do it faster and more elegantly, but you can always do it "somehow"
  - Corollary: Liking programming = liking math

Fuzzy Search 1/4

- Problem setting
  - Given a "dictionary" = a list of "names" of any kind

    For ES5: a list of **2,627,825** entities from Wikidata
  - For a given query, find matching names from that dictionary

Query: frei	Match: Freiburg	**prefix** search
Query: breifurg	Match: freiburg	**fuzzy** search
Query: brei	Match: freiburg	**fuzzy prefix** search
  - We will solve the last two (normal prefix search is a special case); the challenges will be similar as in Lectures 1+2

    Challenge 1: find a good model of what **matches**

    Challenge 2: build an index, so that we find matches **fast**

Fuzzy Search 2/4

- Possible origins for the dictionary
  - Popular queries extracted from a query log

    Basis for Google's query-suggestion feature
  - Words + common phrases from a text collection

    Extracting common phrases from a given text collection is an interesting problem by itself, however, not one we will deal with in this course
  - A list of names of entities

    For example: person names, movie titles, places, street addresses, …

Fuzzy Search 3/4

- Simple solution
  - Iterate over all strings in the dictionary, and for each check whether it matches
  - This is what the Linux commands **grep** and **agrep** do

    grep –x frei.* <file>

    agrep –x –2 breifurg <file>

    agrep –x –2 brei.* <file>

    All matching lines in <file> will be output

    The option –x means match whole line (not just a part)

    The option –2 means allow up to two "errors" … next slide

Fuzzy Search 4/4

- Simple solution, time complexity
  - The time complexity of a tool like **agrep** is $n \cdot T$, where

    n = #records, T = time for checking a single string
  - For fuzzy search, $T \approx 1\mu s$ ... find out yourself in ES5
  - In search, we always want interactive query times

    A response time feels interactive until around **200ms**
  - So the simple solution is fine for up to ≈ 200K records
  - For larger input sets, we need to pre-compute something

    We will build a **q-gram index** (slides 26 – 29) and make use of it in a clever way (slides 18 – 25)

Edit distance 1/7

Vladimir Levenshtein
1935 - 2017

- Definition … aka Levenshtein distance, from 1965
  - Definition: for two strings x and y

    ED(x, y) := minimal number of tra'fo's to get from x to y
  - Transformations allowed are:

    insert(i, c) : insert character c at position i

    delete(i) : delete character at position i

    replace(i, c) : replace character at position i by c

x = DOOF
BOOF ← REPLACE(1, B)
BLOF ← REPLACE(2, L)
BLOEF ← INSERT(4, E)
y = BLOED ← REPLACE(5, D)

IMPORTANT:
this only shows
ED(x, y) ≤ 4

Edit distance 2/7

- Some simple notation
  - The empty word is denoted by ε
  - The length (#characters) of x is denoted by $|x|$
  - Substrings of x are denoted by $x[i..j]$, where $1 \leq i \leq j \leq |x|$
- Some simple properties
  - $ED(x, y) = ED(y, x)$
  - $ED(x, \varepsilon) = |x|$
  - $ED(x, y) \geq abs(|x| - |y|)$ $\quad abs(z) = z \geq 0 \; ? \; z : -z$
  - $ED(x, y) \leq ED(x[1..n-1], y[1..m-1]) + 1$ $\quad n = |x|, m = |y|$

Edit distance 3/7

- Recursive formula
  - For $|x| > 0$ and $|y| > 0$, $ED(x, y)$ is the minimum of

    (1) $ED(x[1..n], y[1..m-1]) + 1$

    (2) $ED(x[1..n-1], y[1..m]) + 1$

    (3a) $ED(x[1..n-1], y[1..m-1]) + 1$ if $x[n] \neq y[m]$

    (3b) $ED(x[1..n-1], y[1..m-1])$ if $x[n] = y[m]$
  - For $|x| = 0$ we have $ED(x, y) = |y|$
  - For $|y| = 0$ we have $ED(x, y) = |x|$

The proof of that formula is not trivial, see "Algorithmen und Datenstrukturen" SS 2021, Lecture 11, slides 13 – 19

Edit distance 4/7

- Algorithm for computing ED(x, y)
  - The recursive formula from the previous slide naturally leads to the following dynamic programming algorithm
  - Takes time and space $\Theta(|x| \cdot |y|)$

		$y \rightarrow$					
		ε	B	L	O	E	D
	ε	0	1	2	$\boxed{3}$	4	5
	D	1	1	2	3	4	$\boxed{4}$
$x \downarrow$	O	2	2	2	2	3	4
	O	3	3	3	2	3	4
	F	4	4	4	3	3	$\boxed{4}$

ED (ε, BLO)

ED (D, BLOED)

ED (DOOF, BLOED)

Edit distance 5/7

ED(uni, u)
ED(uni, un)
E(uni, uni)
...

NOTE: in general
PED(x, y) ≠ PED(y, x)

- **Prefix** edit distance
  - The prefix edit distance between x and y is defined as

    $PED(x, y) = \min_{y'} ED(x, y')$ where y' is a prefix of y
  - For example

    PED(uni, university) = 0 ... but ED = 7

    PED(uniwer, university) = 1 ... but ED = 5
  - Important for fuzzy search-as-you type suggestions

    Nowadays, all the large web search engines have this feature, because it is so convenient for usability

Edit distance 6/7

x = FIBU
y = FREIBURG

- Computation of the PED
  - Compute the entries of the $|x| \cdot |y|$ table, just as for ED
  - The PED is just the minimum of the entries in the last row
  - Important optimization: when $|x| << |y|$ and you only want to know if $PED(x, y) \leq \delta$ for some given δ:

    Enough to compute the first $|x| + \delta + 1$ columns … **verify !**

		ε	F	R	E	I	B	U	R	G
	ε	0	1	2	3	4	5	6	7	8
	F	1								
x ↓	I	2								
	B	3								
	U	4	$\boxed{3}$	3	3	3	3	$\boxed{2}$		

no need to look to the right of this if you just want to know if PED(x,y) ≤ 2

← PED(x,y) is the minimum of the last row

ED(FIBU, F)

ED(FIBU, FREIBU)

Edit distance 7/7

- Implementation of the PED
  - Implementing the PED is fun, but we figured that ES5 is already enough work (and enough to think) without this
  - We therefore provide the implementation for you
  - Since the implementation is critical for the performance, we actually provide two versions

```
ped_python.ped(string1, string2, delta)    # impl. in Python
ped_c.ped(string1, string2, delta)         # impl. in C
```

  - The usage in Python is exactly the same, but ped_c is **much** faster … to install it, you need an additional step:

```
python3 setup.py install --user
```

Alternatively, you can use ped_python with **pypy3**

Fuzzy search with q-grams 1/8

- Motivation
  - Find fuzzy matches for a given word x in a dictionary D faster than by computing ED(x, y) or PED(x, y) for **all** $y \in D$
  - Intuitive idea: when two strings have a small edit distance, they overlap in many of their parts

    Example 1: is ED("freiburg", "stuttgart") small ?

    No, the strings are very different "everywhere"

    Example 2: is ED("freiburg", "breifurg") small ?

    Yes, the strings are similar "at many places"
  - On the following slides, we will make this idea more precise and turn it into an efficient algorithm

Fuzzy search with q-grams 2/8

- Definition of a q-gram
  - For a string x, and an integer $q \in \mathbf{N}$, the multiset of q-grams, denoted by $\mathbf{Q_q(x)}$, consists of all substrings of length q

    Q_3("freiburg") = { "fre", "rei", "eib", "ibu", "bur", "urg" }

    We define it as a **multi**set because the same q-gram may occur multiple times and we want to know when it does

    Q_3("ababa") = { "aba", "bab", "aba" }
  - The number of q-grams of a string x is:

    $|Q_q(x)| = |x| - q + 1$

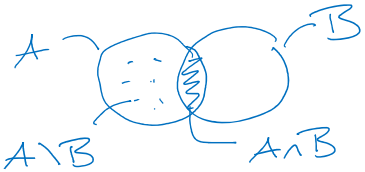
x = freiburgerin
y = urgenumbrei

Fuzzy search with q-grams 3/8

- Similar words have many q-grams in common
  - **Lemma:** for strings x and y: $|Q_q(x) \setminus Q_q(y)| \leq q \cdot ED(x, y)$

    Understand: $A \setminus B$ denotes the **set difference**, that is, the elements of A without the elements from B

    If B is very similar to A, then $A \setminus B$ is small



$|A \setminus B| = |A| - |A \cap B|$

  - Example:

    x = freiburgerin, y = breifurgerin, ED(x, y) = 2

    $Q_2(x)$ = { fr, re, ei, ib, bu, ur, rg, ge, er, ri, in } $|Q_2(x)| = 11$

    $Q_2(y)$ = { br, re, ei, if, fu, ur, rg, ge, er, ri, in } $|Q_2(y)| = 11$

    $Q_2(x) \setminus Q_2(y)$ = { fr, ib, bu }

    $|Q_2(x) \setminus Q_2(y)|$ = 3 ≤ 4

    what the lemma says

Fuzzy search with q-grams 4/8

$q = 3$
$x = \text{FREIBURG}$
$y = \text{FREIXURG}$

- Similar words have many q-grams in common
  - **Lemma:** for strings x and y: $|Q_q(x) \setminus Q_q(y)| \leq q \cdot ED(x, y)$
  - **Proof:** For $ED(x, y) = 1$, one edit distance operation can "affect" at most q q-grams, hence $|Q_q(x) \setminus Q_q(y)| \leq q$

    For the general case $ED(x, y) = k$, let $x = x_0 \rightarrow \ldots \rightarrow x_k = y$ be a sequence of k ED operations that turn x into y

    That is, $ED(x_{i-1}, x_i) = 1$ and therefore $|Q_q(x_{i-1}) \setminus Q_q(x_i)| \leq q$

    It remains to prove that the following holds for any sequence $A_0, \ldots, A_k$ of sets: $A_0 \setminus A_k \subseteq \bigcup_{i=1..k} (A_{i-1} \setminus A_i)$

	A_0	A_1	A_2	$A_3 \ldots$	A_q
$x \in ?$	✓	✓	✓	×	×

$x \in A_0 \setminus A_q$

$\Rightarrow \exists i: x \in A_{i-1}, x \notin A_i$

$\Rightarrow x \in A_{i-1} \setminus A_i \Rightarrow x \in \bigcup_{i=1}^{q} A_{i-1} \setminus A_i$ ∎

Fuzzy search with q-grams 5/8

- Similar words have many q-grams in common
  - **Corollary 1:** for arbitrary strings x and y

    $|Q_q(x) \cap Q_q(y)| \geq \max\{ |Q_q(x)|, |Q_q(y)| \} - q \cdot \mathbf{ED}(x, y)$
  - **Example:**

    $= 8$, see slide 20

    $|Q_2(\text{freiburgerin}) \cap Q_2(\text{breifurgerin})| \geq 11 - 2 \cdot 2 = 7$
  - **Proof:**

    $|A \setminus B| = |A| - |A \cap B|$

    For arbitrary sets A and B, $|A \setminus B| = |A| - |A \cap B|$, hence

    $|Q_q(\mathbf{x})| - |Q_q(x) \cap Q_q(y)| = |Q_q(x) \setminus Q_q(y)| \leq q \cdot ED(x, y)$

    Since ED(x, y) = ED(y, x), the same lemma also gives us

    $|Q_q(\mathbf{y})| - |Q_q(x) \cap Q_q(y)| = |Q_q(y) \setminus Q_q(x)| \leq q \cdot ED(x, y)$

    The two inequalities together give us the corollary

Fuzzy search with q-grams 6/8

- Similar words have many q-grams in common
  - **Corollary 2:** for arbitrary strings x and y

    $|Q_q(x) \cap Q_q(y)| \geq |Q(x)| - q \cdot \mathbf{PED}(x, y)$
  - **Example:**

    $|Q_2(\text{frei}) \cap Q_2(\text{breifurg})| \geq 3 - 2 \cdot 1 = 1$
  - **Proof sketch:**

    $PED(x, y) = ED(x, y')$ for some prefix y' of y

    Like on the previous slide:

    $|Q_q(\mathbf{x})| - |Q_q(x) \cap Q_q(y')| = |Q_q(x) \setminus Q_q(y')| \leq q \cdot ED(x, y')$

    The corollary follows because $Q_q(y) \supseteq Q_q(y')$

    Note that in general, PED(x, y) is **not** equal to PED(y, x)

Fuzzy search with q-grams 7/8

- Algorithm for ED … for some fixed $q \in \mathbf{N}$
  - Problem: Given x and δ, find all $y \in D$ with $\mathbf{ED}(x, y) \leq \delta$
  - For each word $y \in D$, do the following:
    1. Is $|Q_q(x) \cap Q_q(y)| \geq \max\{ |Q_q(x)|, |Q_q(y)| \} - q \cdot \delta$?
    2. NO: We know that $ED(x, y) > \delta$ and y is not a match
    3. YES: Compute $ED(x, y)$ to determine if y is a match
- Algorithm for PED … for some fixed $q \in \mathbf{N}$
  - Problem: Given x and δ, find all $y \in D$ with $\mathbf{PED}(x, y) \leq \delta$
  - Just like the above, but using the bound from Corollary 2 instead of the bound from Corollary 1

Fuzzy search with q-grams 8/8

- Improvement for ED
  - **Pad** all strings in D with q – 1 times $ on **both** sides

    For example (q = 3): freiburg → $$freiburg$$

    The algorithm then gives the same result because:

    x' and y' padded version of x and y → ED(x, y) = ED(x', y')

    However, with the padding we get more q-grams and hence Step 2 can be applied more often
- Improvement for PED
  - **Pad** all strings in D with q – 1 times $ on the **left** side only

    For example (q = 3): freiburg → $$freiburg

    x'' and y'' padded versions → PED(x, y'') = PED(x, y'')

Q-gram index 1/4

- Motivation
  - For the algorithm described on slide 24, we need to compute, for a given x, $|Q_q(x) \cap Q_q(y)|$ for all $y \in D$

    For ES5, that's 2.6 million strings
  - How do we do that efficiently?

    The answer is: with a **q-gram index** … see next slides

Q-gram index 2/4

- Definition
  - For each q-gram of a string from D, store an inverted list of all strings from D containing it, sorted lexicographically

    $fr : **fr**ankfurt, **fr**eiburg, **fr**eetown, **fr**esno, …

    ibu : chic**ibu**, fre**ibu**rg, **ibu**suki, mal**ibu**, …

    As usual, store **ids** of the strings, not the strings themselves
  - A q-gram index for a collection of words is also an inverted index, just with different objects than a document index:

    document index : lists of **doc ids**, one for each **word**
    q-gram index : lists of **words ids**, one for each **q-gram**
  - Let's adapt our code from Lecture 1 to q-grams, to see how similar the code is …

Q-gram index 3/4

- Using the q-gram index for our algorithm
  - Given x, we want to compute $|Q_q(x) \cap Q_q(y)|$ for all $y \in D$
  - Proceed as follows:
    1. Compute $Q_q(x)$, the set of q-grams of the given x
    2. For each q-gram from $Q_q(x)$, fetch the inverted list
    3. Compute the union of these $|Q_q(x)|$ inverted lists, and count how many times each word occurs

       Note: if a word y occurs k times in the union, it means that it contains exactly k q-grams from $Q_q(x)$, and hence that $|Q_q(x) \cap Q_q(y)| = k$

       For all words y that are not encountered in the union, we know that $|Q_q(x) \cap Q_q(y)| = 0$

Q-gram index 4/4

- Example: x = brei, $\delta = 1$ … using a 2-gram index
  - Fetch the following inverted lists ($brei → $b, br, re, ei)

    $b : bangalore, beijing, berlin, …
    br : brisbane, brussels, gibraltar, …
    re : bangalore, freiburg, singapore, …
    ei : beijing, freiburg, marseille, …

  - According to Corollary 2, for a y with $PED(x, y) \leq \delta = 1$, $|Q_2(x) \cap Q_2(y)|$ must be **at least** $|Q_2(x)| - q \cdot \delta = 2$
  - If we merge the inverted lists above, these three words occur (at least) twice: bangalore, beijing, freiburg
  - We need to compute the PED only for these three words:

    NO MATCH MATCH MATCH

    PED(brei, bangalore) = 3, PED(brei, beijing) = 1, PED(brei, freiburg) = 1

References

- Textbook

  Section 3: Tolerant Retrieval, in particular:

  Section 3.2: Wildcard queries

  Section 3.3: Spelling correction

- Wikipedia

  http://en.wikipedia.org/wiki/N-gram

  http://en.wikipedia.org/wiki/Approximate_string_matching

  http://en.wikipedia.org/wiki/Levenshtein_distance